Laboratory 1

Variant 1

Group 16

Leonardo Meloni and Anatoli Abenet Abahumna

Introduction

The objective of this laboratory was to solve a maze represented as a 2D grid using two search algorithms: Breadth-First Search (BFS) and Depth-First Search (DFS). In the maze, `0` represents an empty cell and `1` represents a wall. The program receives a maze, a starting position, and an ending position. It must return the number of steps from the start to the finish, or `-1` if no path exists. It must also provide a step-by-step visualization of the search process, including all visited cells, not only the final path.

The implementation is contained in `lab1_cg102_g16_v1_Meloni_Abahumna.py`. The final version preserves the required function signatures:

- `bfs(maze, start, finish)`
- `dfs(maze, start, finish)`
- `vizualize(viz)`

Algorithms used in the task

Breadth-First Search (BFS) explores the maze level by level. It first checks all cells that are one move away from the start, then all cells two moves away, and so on. In an unweighted grid such as this maze, BFS always finds the shortest path if a path exists. Its main advantage is correctness for shortest-path problems. Its main disadvantage is that it may visit many cells and use more memory because it stores the frontier in a queue.

Depth-First Search (DFS) explores one branch as deeply as possible before backtracking. It uses less structure than BFS and can reach the finish quickly in some mazes if the exploration order is favorable. Its main disadvantage is that it does not guarantee the shortest path. The result depends strongly on the order in which neighboring cells are explored.

For this assignment, BFS is the more appropriate algorithm when the goal is the minimum number of steps. DFS is useful for comparison because it shows how search order affects both the explored cells and the final path.

Implementation

Input validation

Before the search starts, the program validates the maze shape and the coordinates. This prevents crashes on empty, non-rectangular, or invalid-coordinate inputs and makes the solver more robust on hidden test cases.

```
def validate_maze_input(maze, start, finish):
    if not isinstance(maze, list) or len(maze) == 0:
        return False, 0, 0

    if not all(isinstance(row, list) for row in maze):
        return False, 0, 0

    cols = len(maze[0])
    if cols == 0:
        return False, 0, 0

    if any(len(row) != cols for row in maze):
        return False, 0, 0
```

The validation also checks whether `start` and `finish` are valid coordinates and rejects negative or out-of-bounds positions. A limitation of the current implementation is that it does not verify that every maze cell is strictly `0` or `1`; it therefore assumes that the maze contents follow the assignment format. When the input is invalid, both algorithms return `-1` together with a visualization dictionary in which `visited_order` and `path` are empty.

How BFS works in the program

The BFS implementation uses a queue created with `collections.deque`. A `visited` set prevents revisiting cells, and a `parent` dictionary stores the previous cell for each discovered position so the path can be reconstructed later.

```
queue = deque([(start, 0)])
visited = {start}
visited_order = []
parent = {start: None}

while queue:
    current, steps = queue.popleft()
    visited_order.append(current)
```

The execution flow is:

1. Validate the maze and the coordinates.
2. Reject the input if the start or finish is inside a wall.
3. Insert the start cell into the queue.
4. Repeatedly remove the first cell from the queue.
5. Add each valid unvisited neighbor to the end of the queue.
6. Store the parent of each visited neighbor.
7. When the finish is reached, reconstruct the path and return `len(path) - 1`.

Because BFS expands by distance from the start, the first time the finish is removed from the queue, the path is guaranteed to be the shortest one.

How DFS works in the program

The DFS implementation is similar, but it uses a stack based on a Python list.

```
stack = [(start, 0)]
visited = {start}
visited_order = []
parent = {start: None}

while stack:
    current, steps = stack.pop()
    visited_order.append(current)
```

The execution flow is:

1. Validate the maze and the coordinates.
2. Reject invalid inputs and wall positions.
3. Push the start cell onto the stack.
4. Repeatedly remove the last cell from the stack.
5. Push every valid unvisited neighbor onto the stack.
6. Use the same `parent` mechanism to reconstruct the path when the finish is found.

The algorithm uses the direction order `up, down, left, right`. Since DFS uses a stack, the last valid neighbor added is explored first. This detail is important because it changes the path that DFS follows and sometimes makes the DFS result longer than the BFS result.

Common Traversal and Path Reconstruction Logic

The core mechanism for checking the destination and exploring neighboring cells is shared completely between both algorithms. The only distinction in this phase lies in how new valid cells are stored: using `stack.append()` for DFS versus `queue.append()` for BFS.

During each iteration, the algorithm checks if the `current` node is the destination. If it is, it triggers the `get_path` helper to reconstruct the final route and returns the total steps. If not, it iterates through the four possible `directions`, calculating the new coordinates `(nr, nc)`.

Before adding a neighbor to the frontier, it performs a strict validation: it ensures the coordinates are within the maze boundaries, the cell is not a wall (`maze[nr][nc] == 0`), and it has not been explored yet. Valid cells are then added to the `visited` set, their `parent` is recorded, and they are pushed to the data structure.

```
if current == finish:
    path = get_path(parent, finish)
    return len(path) - 1, build_viz(
        maze, start, finish, visited_order, path, "DFS"
    )

r, c = current
for dr, dc in directions:
    nr, nc = r + dr, c + dc

    if 0 <= nr < rows and 0 <= nc < cols and maze[nr][nc] == 0:
        if (nr, nc) not in visited:
            visited.add((nr, nc))

            # This is the only line that differs (queue.append for BFS)
            stack.append(((nr, nc), steps + 1))
            parent[(nr, nc)] = current
```

Visualization

The visualization is console-based. During the search:

- `S` marks the start
- `E` marks the finish
- `@` marks the current cell being processed
- `.` marks previously visited cells
- `###` marks walls

The program first replays the exploration order stored in `visited_order`, then prints the final path with `*` when a path exists. The visualization also handles invalid or empty mazes without crashing.

Visualizations generated by the code

The following frames come from the comparison maze included in the project:

```
Maze:
[0, 1, 0, 0, 0]
[0, 0, 0, 1, 0]
[0, 0, 1, 1, 0]
[0, 0, 0, 0, 0]

Start = (0, 0)
Finish = (3, 4)
```

BFS, step 8

```
 S ###
 .  .  . ###
 .  . ######
 .  @       E
```

At this point BFS has spread through the lower part of the maze and is still exploring by distance from the start. The current cell is near the bottom corridor, which is part of the shortest route to the exit.

BFS, final path

```
 S ###
 *      ###
 *    ######
 *  *  *  *  E
```

This is the shortest path found by BFS. The solver returns `7` steps.

DFS, step 8

```
 S ### .  .  .
 .  .  . ### @
      ######
            E
```

Here DFS has committed to the upper corridor because of the stack behavior and the current neighbor order. It reaches the right side of the maze quickly, but it does so through a longer route.

DFS, final path

```
S ### *  *  *
*  *  * ### *
     ###### *
            E
```

DFS reaches the finish, but the path length is 9 steps, which is longer than the BFS path.

Discussion

Results obtained from the implemented tests

Test case	BFS		DFS		Observation
	Steps	Visited	Steps	Visited	
Complex maze	6	8	6	32	Same path length, but DFS explored far more cells.
Open space	7	20	7	16	Same path length. DFS happened to reach the finish with fewer visited cells.
No path available	-1	4	-1	4	Both algorithms correctly reported that no path exists.
Start equals finish	0	1	0	1	Both algorithms correctly returned zero steps.
Start inside a wall	-1	0	-1	0	Both algorithms rejected the maze immediately because the start cell is blocked.
Finish inside a wall	-1	0	-1	0	Both algorithms rejected the maze immediately.
1x1 open maze	0	1	0	1	Both algorithms handled the smallest valid maze.
1x1 blocked maze	-1	0	-1	0	Both algorithms correctly rejected the blocked cell.
Disconnected maze	-1	1	-1	1	The start cell is valid, but no movement is possible, so both searches stop immediately.
BFS shorter than DFS	7	14	9	10	BFS found the shorter path, while DFS found a longer one.
Empty maze	-1	0	-1	0	Added as a robustness test.
Non-rectangular maze	-1	0	-1	0	Added as a robustness test.
Out-of-bounds start	-1	0	-1	0	Invalid coordinates are rejected before the search starts.
Negative coordinate	-1	0	-1	0	Negative indices are rejected before the search starts.

Table 1: Measured results produced by the current implementation.

Differences between BFS and DFS in this project

The main difference is that BFS guarantees the shortest path, while DFS does not. This can be seen clearly in the comparison maze:

- BFS returned `7` steps.
- DFS returned `9` steps.

This happened because BFS explores all positions at distance 1, then all positions at distance 2, and so on. DFS instead follows one branch deeply. In this project, the fixed direction order made DFS follow the upper corridor first, which produced a valid but longer path.

There are also cases where both algorithms return the same number of steps but do a very different amount of work. In the complex maze, BFS and DFS both returned `6` steps, but BFS visited only `8` cells while DFS visited `32`. This shows that DFS can spend a large amount of time exploring branches that do not help reach the finish efficiently.

The open-space case shows another important point. DFS is not always worse in terms of explored cells. In that test, DFS visited `16` cells, while BFS visited `20`. The path length was still the same for both algorithms. This happened because the direction order guided DFS toward the finish without much backtracking. Even so, this does not change the main property of the algorithms: only BFS guarantees the shortest path in every unweighted maze.

Test cases that show strengths and weaknesses

Complex maze. This test contains branches and obstacles. It shows the strength of BFS in structured mazes where the finish can be reached at a shallow search depth. DFS found a path of the same length, but it explored many more cells first.

Open space. This test shows that DFS can sometimes perform well if the exploration order points toward the finish. In this case both algorithms found a path of length `7`, and DFS visited fewer cells than BFS.

Comparison maze. This is the strongest test for showing the difference between the algorithms. BFS found the shortest path in `7` steps, while DFS found a longer path of `9` steps. This demonstrates the main weakness of DFS for shortest-path problems.

No path available. This confirms that both algorithms stop correctly and return `-1` when the finish is unreachable.

1x1 mazes and wall cases. These cases verify that the implementation handles corner cases correctly: immediate success, immediate failure, and blocked endpoints.

Empty and non-rectangular mazes. These were added as robustness tests. They are beyond the original maze definition, but they prevent the program from crashing on malformed input.

Conclusion

This laboratory showed clearly how the choice of search algorithm changes the result of solving a maze. BFS is the correct algorithm when the objective is the minimum number of steps, because it always finds the shortest path in an unweighted grid. DFS can still find a valid path, but the path may be longer and the result depends on the order in which neighbors are explored.

The most important result of the project is the comparison maze, where BFS returned `7` steps and DFS returned `9`. This directly shows why BFS is preferred for shortest-path problems. The complex maze also demonstrated that even when both algorithms return the same path length, BFS may still be much more efficient in terms of explored cells.

The main difficulty in the implementation was combining three requirements at the same time: correct path search, storing every visited cell for visualization, and handling corner cases without breaking the required function signatures. Another important point was to keep the visualization simple while making sure it showed the actual exploration process rather than only the final path.

The project can still be improved further. A next step would be to separate testing from visualization more clearly, add automatic assertions for expected outputs, and make the visualization speed configurable. It would also be useful to compare more neighbor orders in DFS to show more clearly how sensitive DFS is to traversal order.